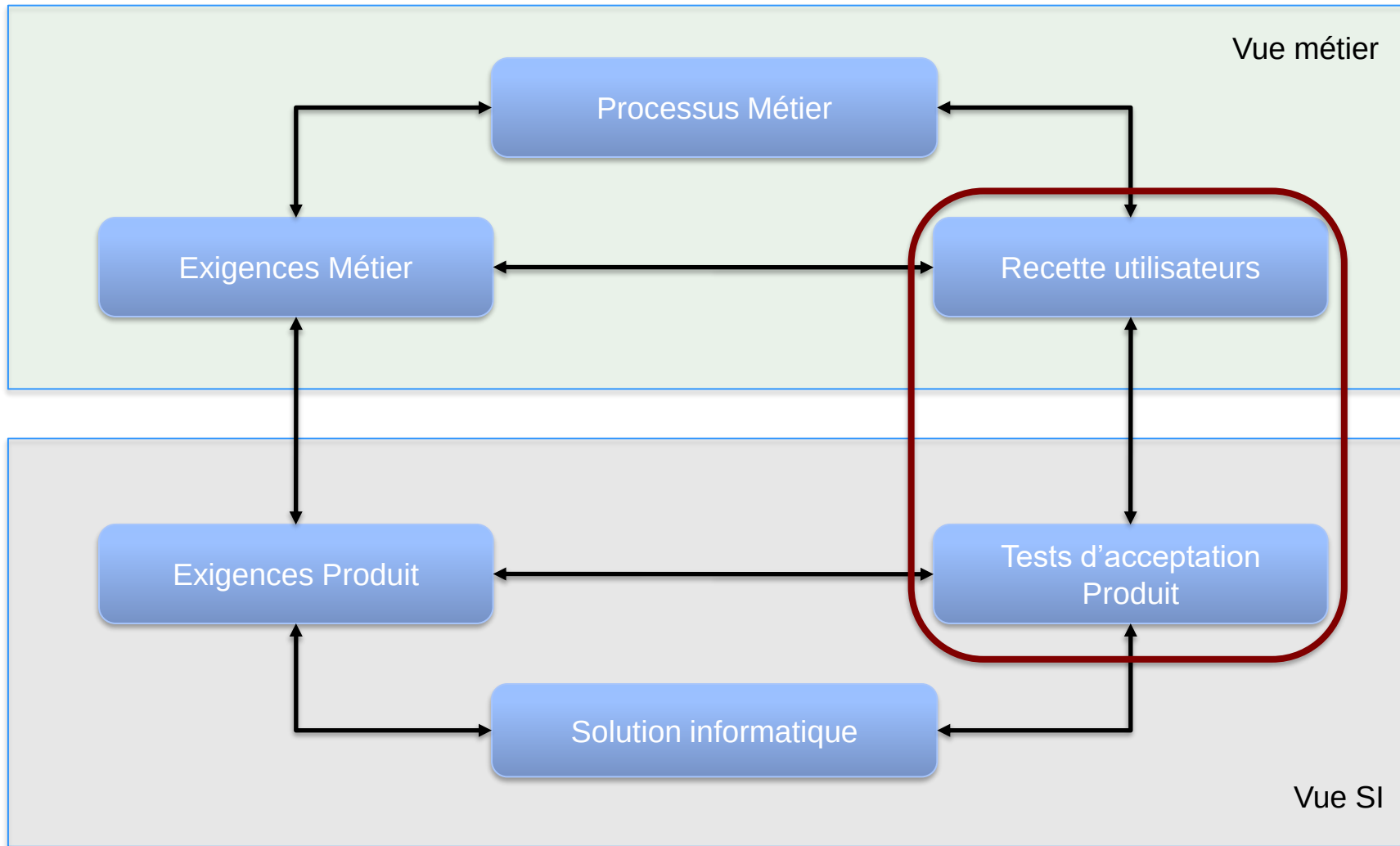


Formation AMOA

**La recette métier : des tests d'acceptation
produit à la recette utilisateur**

Contexte



1. Quoi

- 1.1. Objectifs et enjeux
- 1.2. Contextes opérationnels
- 1.3. Tests de qualification et tests métier
- 1.4. Principales activités et livrables

2. Comment

- 2.1. Construire un plan de test
- 2.2. Conduire une campagne de tests
- 2.3. Gérer les anomalies
- 2.4. Rédiger un rapport de tests métier

1. Quoi

1.1. Objectifs et enjeux

1.2. Contextes opérationnels

1.3. Tests de qualification et recette métier

1.4. Activités et livrables



1.1. Objectifs et enjeux

Objectifs

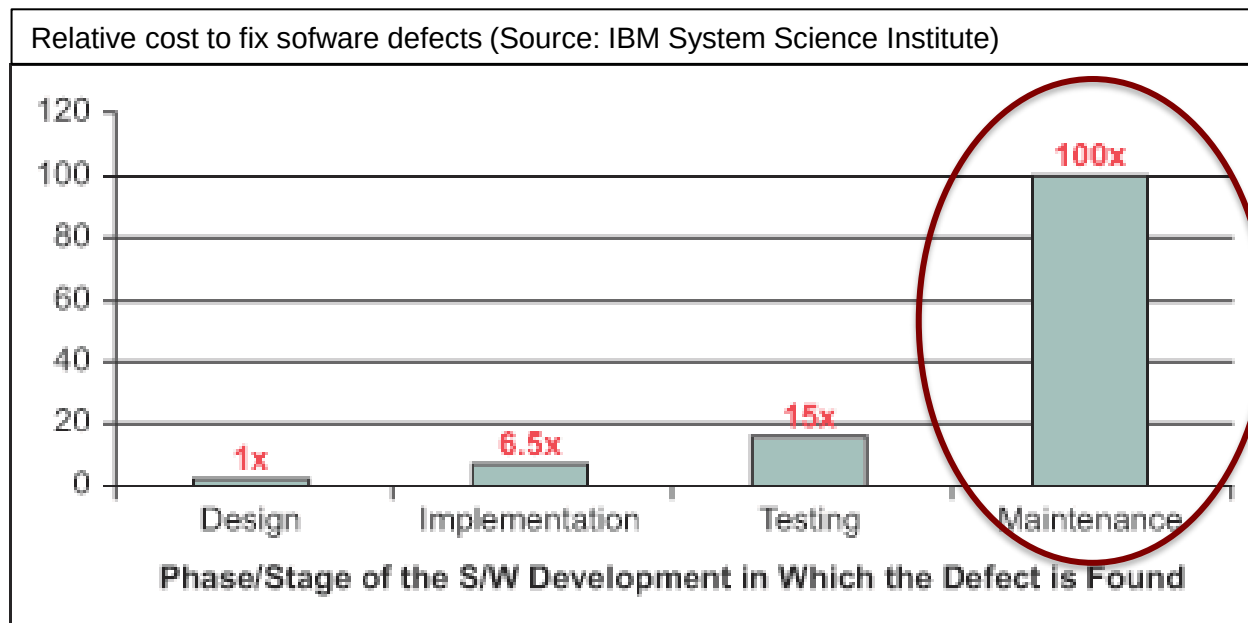
- **Objectif : livrer aux acteurs métier la solution SI la plus adaptée à un usage opérationnel**
 - **Résoudre** avant usage opérationnel **toutes** les anomalies SI ayant **des impacts très significatifs sur le métier**
 - **Définir des contournements et établir un plan de livraison** pour les anomalies SI ayant un impact significatif sur le métier et **ne pouvant être réparées** avant usage opérationnel
 - **Communiquer** avant usage opérationnel les impacts des anomalies résiduelles

Comment être certain que toutes les anomalies significatives ont été résolues ou ont une date de résolution prévue ?

1.1. Objectifs et enjeux

Enjeux

- **Livrer** une solution SI qui correspond à l'objectif métier et assure les gains attendus (financiers, sécurité, réglementation...)
- **Réduire** le coût des anomalies relevées après le Cut Over (incident)



Source: <http://www.isixsigma.com/industries/software-it/defect-prevention-reducing-costs-and-enhancing-quality/>

1. Quoi

1.1. Objectifs et enjeux

1.2. Contextes opérationnels

1.3. Tests de qualification et recette métier

1.4. Activités et livrables

1.2. Contextes opérationnels

Contextes	Questions
Evolution majeure d'une application critique	• Comment s'assurer de la non régression des fonctions critiques de l'application ? • <i>Tests de non-régressions critiques (effectués ou non par l'IT)</i> • Comment gérer les anomalies résiduelles après les tests ? • <i>Contournements et planning de livraison des réparations.</i>
Moteur de recherche opérationnel	• Comment valider la conformité des réponses de l'outil ? • <i>Vérifier avec des utilisateurs clés, outils de mesure</i>
Solution multi-composants	• Comment s'organiser avec d'autres AMOA ? • <i>Se mettre d'accord en amont sur qui fait quoi et quand</i>
Montée de version de progiciel	• Comment vérifier la non-régression ? Quels aspects devraient être vérifiés ? • <i>Non-régression sur les customisations, les interfaces, les fonctions critiques</i>
Evolution réglementaire	• Comment valider le complet respect des règles ? • <i>Tout dépend de la criticité des règles (effort de test)</i>
Tests en production	• Dans quels cas exceptionnels des tests peuvent-ils être exécutés en production ? • <i>En cas de marche en double et de fonctions non testables en environnement de test</i>

La phase d'acceptation peut être **courte** (moins d'un mois) ou **longue** (plus de 6 mois)

1. Quoi

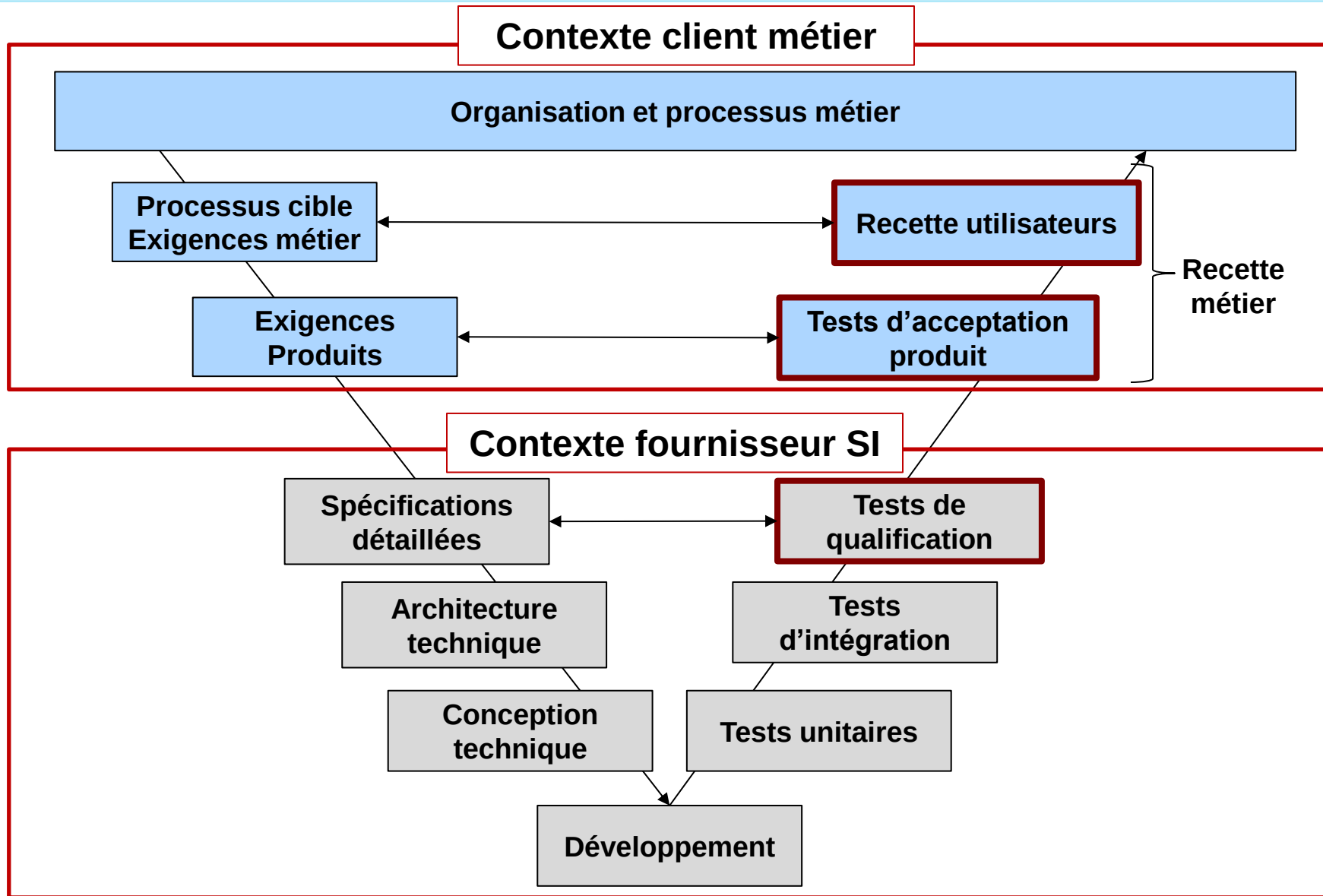
1.1. Objectifs et enjeux

1.2. Contextes opérationnels

1.3. Tests de qualification et recette métier

1.4. Activités et livrables

1.3. Tests de qualification et recette métier (1/3)



1.3. Tests de qualification et recette métier (2/3)

	Tests de qualification	Recette métier
Objectif	• Livrer à l'AMOA un produit SI en accord avec les spécifications	• Livrer aux acteurs métier la bonne solution SI pour une utilisation opérationnelle
Périmètre	• Une application	• La solution globale dans son contexte métier et SI
Résultats attendus	• Absence d'anomalies critiques (incident technique) • Anomalies majeurs résolues ou à minima remontées	• Absence d'anomalies hautement significatives • Plan de livraison pour les anomalies significatives résiduelles
Livrables de référence	• Spécifications fonctionnelles détaillées	• Exigences • Cible métier
Point de sortie	• Livraison sur l'environnement d'acceptation	• GO pour un usage opérationnel

Comment s'assurer que la résolution de chaque anomalie significative a été faite ou est planifiée ?

1.3. Tests de qualification et recette métier (3/3)

- L'AMOA pilote et coordonne la **recette métier** et **est informé des résultats des tests de qualification**
- Plus les tests de qualification sont **pertinents et fructueux**, plus les tests métier sont **sécurisés et rapides**.
- La phase de tests métier est **la dernière étape avant l'usage opérationnel d'une nouvelle solution SI**

1. Quoi

1.1. Objectifs et enjeux

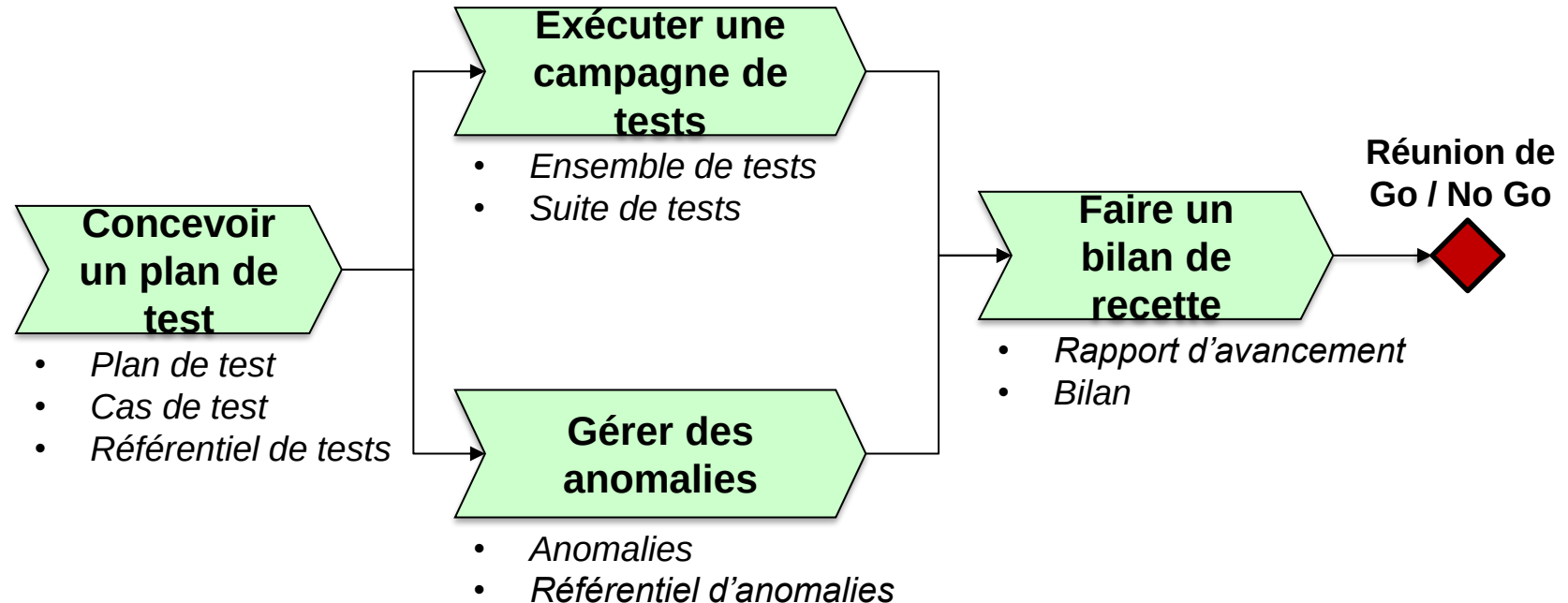
1.2. Contextes opérationnels

1.3. Tests de qualification et recette métier

1.4. Activités et livrables



1.4. Activités et livrables



**Tests de qualification logicielle et recette métier:
Activités principales et livrables identiques**

2. Comment?

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2. Comment?

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.1.1. Définir quoi tester

2.1.2. Définir le périmètre de test

2.1.3. Concevoir les cas de test

2.1.4. Contrôler les ressources de test

2.1.5. Formaliser un plan de test

2.1.1. Définir quoi tester

Tests de qualification : les spécifications fonctionnelles

2.1. Concevoir un plan de test

Objectif : définir le périmètre et les priorités des tests de qualification

Quoi	Pourquoi	Comment
Extraire des exigences de test des spécifications	Point d'entrée pour le cadrage des tests	Recenser toutes les règles, données et interfaces décrites dans les spécifications
Prioriser les exigences de test	Point d'entrée pour le cadrage des tests	- Valider la criticité des exigences concernées - Valider la complexité du développement
Valider les impacts des nouvelles spécifications	Point d'entrée pour le cadrage des tests (non régression)	Valider les fonctions existantes

Approche Agile : moins de spécifications, plus de tests

2.1.1. Définir quoi tester

Tests d'acceptation produit : les exigences produit

2.1. Concevoir un plan de test

Objectif : définir le périmètre et les priorités des tests métier

Quoi	Pourquoi	Comment
Recenser toutes les nouvelles exigences unitaires	Point d'entrée pour le cadrage des tests	- Valider les livrables d'exigences - Fractionner les exigences en exigences unitaires (si cela n'a pas déjà été fait)
Valider les propriétés des nouvelles exigences unitaires	Point d'entrée pour le cadrage des tests	- Valider la testabilité - Valider la criticité métier
Valider les impacts des nouvelles exigences	Point d'entrée pour le cadrage des tests	Valider les exigences existantes

Formaliser correctement les exigences facilite cette tâche

2.1.1. Définir quoi tester

Recette utilisateurs: le processus cible

2.1. Concevoir un plan de test

Objectif : améliorer la pertinence des tests métier

Quoi	Pourquoi	Comment
Obtenir des statistiques métier	Constituer des jeux de données pertinents pour les cas de test	- Demander les statistiques existantes - Construire des statistiques
Obtenir des cas d'utilisation métier	Elaborer des scénarios de test pertinents	Valider la cible métier
Valider les rôles des acteurs métier	Elaborer des procédures pertinentes avec les véritables droits des utilisateurs	Valider la cible métier

Bien formaliser et mettre à jour la cible métier facilite cette tâche

2. Comment?

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.1.1. Définir quoi tester

2.1.2. Définir le périmètre de test

2.1.3. Concevoir les cas de test

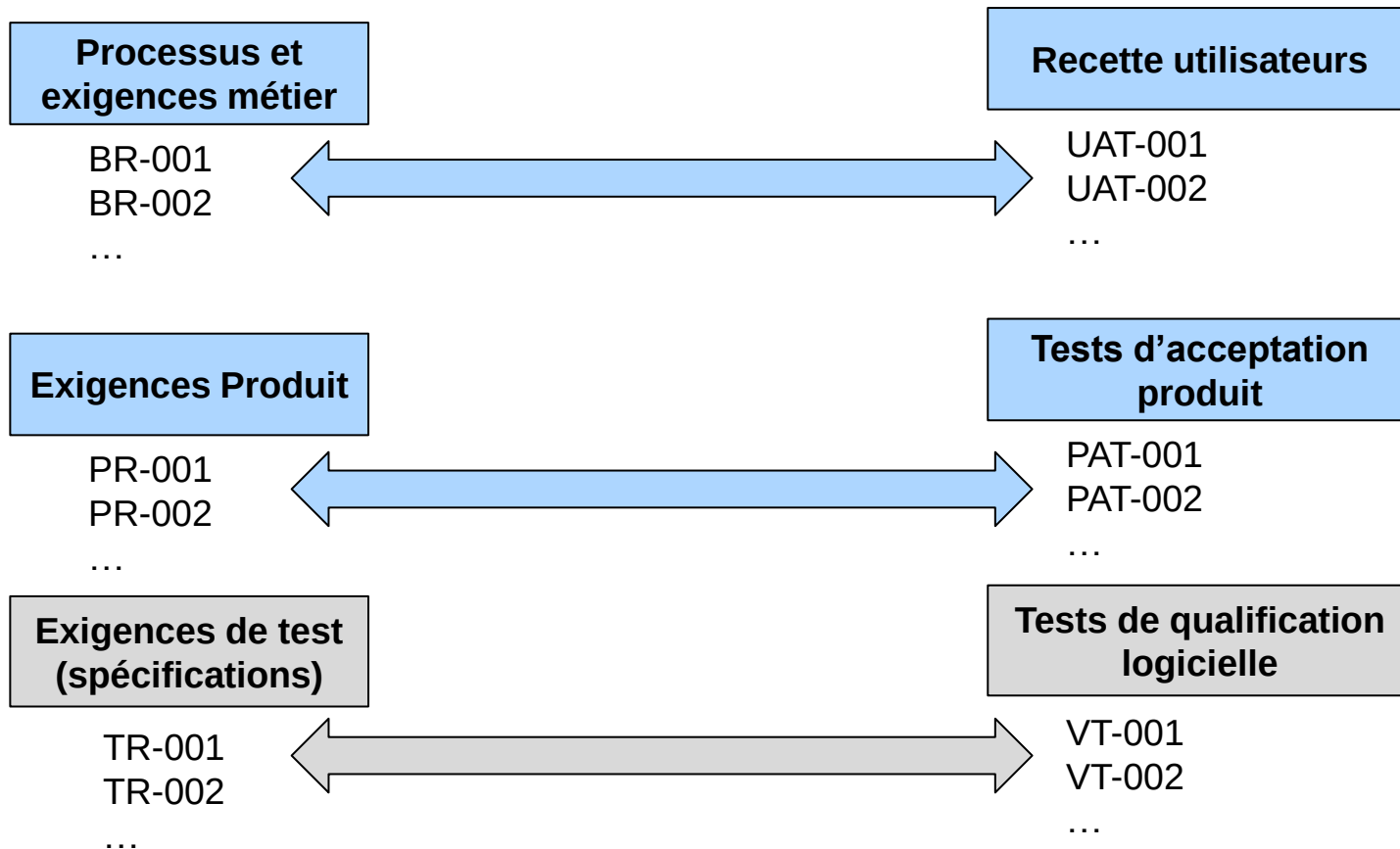
2.1.4. Contrôler les ressources de test

2.1.5. Formaliser un plan de test

2.1.2. Définir le périmètre de test

Définir un périmètre par niveau de test

2.1. Concevoir un plan de test



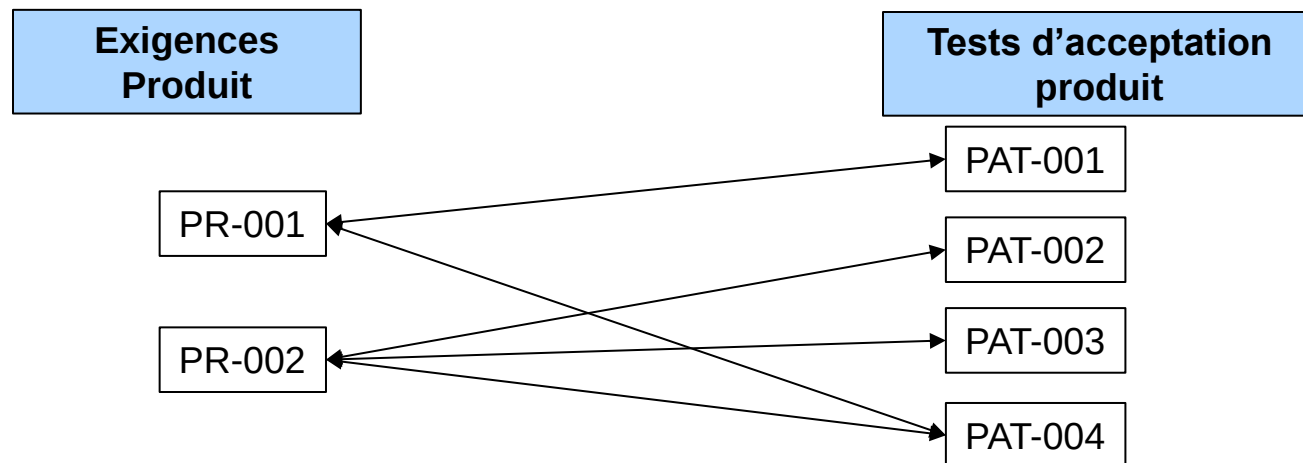
- Chaque niveau doit avoir sa propre référence pour les tests
- **Rappel** : les exigences de test sont extraites des spécifications et ne doivent pas être confondues avec les exigences métier/produit

2.1.2. Définir le périmètre de test

Couverture des exigences par les cas de test (1/2)

2.1. Concevoir un plan de test

- **Objectif : couvrir chaque exigence par au moins un cas de test**
- **Comment : établir le lien entre cas de test et exigences (leur objectif)**
 - 1 cas de test a un et un seul objectif
 - 1 cas de test couvre une exigence en partie ou en totalité
 - 1 cas de test peut couvrir plusieurs exigences



Objectif de test: ce qu'un test valide pour les exigences qu'il couvre

2.1.2. Définir le périmètre de test

Couverture des exigences par les cas de test (2/2)

2.1. Concevoir un plan de test

- **Bonnes pratiques**
 - Pour identifier un cas de test, **ne pas détailler** (prérequis, jeux de données, pas de test), indiquer uniquement l'objectif du cas de test
 - L'objectif de test doit **permettre d'identifier les anomalies significatives**
 - L'objectif de test doit être **clair** et **sans ambiguïté**

2.1.2. Définir le périmètre de test

Construire une matrice de couverture des exigences

2.1. Concevoir un plan de test

- **Objectif : vérifier la pertinence de la matrice de couverture**
- **Comment : construire une matrice de couverture des exigences**

		Test Case 001	Test Case 002	Test Case 003	Test Case 004	Test Case 005	Test Case 006	Test Case 007	Test Case 008	Test Case 009	Test Case 010	Test Case 011	Test Case 012	Test Case 013	Test Case 014	Test Case 015	Test Case 016	Test Case 017
	Criticality																	
Requirement 001	High	X	X	X										X		X		
Requirement 002	High				X	X								X				
Requirement 003	High						X	X							X			
Requirement 004	High							X								X		
Requirement 005	High								X						X			
Requirement 006	Medium									X				X				
Requirement 007	Medium										X							
Requirement 008	Medium											X		X				
Requirement 009	Medium												X					
Requirement 010	Low														X			
Requirement 011	Low															X		
Requirement 012	Low													X				

- **Bonnes pratiques**
 - Les exigences critiques doivent être couvertes par des cas de test dédiés

2. Comment?

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.1.1. Définir quoi tester

2.1.2. Définir le périmètre de test

2.1.3. Concevoir les cas de test

2.1.4. Contrôler les ressources de test

2.1.5. Formaliser un plan de test

2.1.2. Concevoir les cas de test

Modèle de cas de test

2.1. Concevoir un plan de test

Objectif du test

- Couverture d'exigences
- Un et un seul objectif par test

Prérequis

- Conditions fonctionnelles ou techniques à remplir pour démarrer le test
- Le résultat d'un précédent cas de test peut être une telle condition

Jeux de données

- Des données pertinentes traitées de la même manière de façon à obtenir le même résultat

Scénario
Pas 1 – résultat intermédiaire
Pas 2 – résultat intermédiaire
Pas n – résultat intermédiaire

- Enchaînement d'actions à effectuer pour obtenir le résultat attendu
- En production, les actions peuvent être effectuées par un ou plusieurs acteurs métier

Résultat(s) attendu(s)

- Le résultat attendu ne doit souffrir d'aucune ambiguïté (OK ou KO)
- Le(s) résultat(s) attendu(s) correspond(ent) à l'objectif de test

2.1.2. Concevoir les cas de test

Jeux de données (1/2)

2.1. Concevoir un plan de test

- **Objectif : définir des jeux de données pertinents pour les cas de test**
- **Comment**

Données	Tests de qualification	Tests métier
Valeurs justes	• Valeurs possibles • Valeurs aux limites	• Vraies valeurs (statistiques) • Valeurs particulièrement critiques (analyse de risque)
Valeurs fausses	• Mauvais format • Attribut faux ou manquant	• Erreur humaine
Quantité	• N/A	• Volume de données de production

- **Bonnes pratiques**
 - La cible métier définira les jeux de données pertinents pour les tests métier
 - Construire des jeux de données stables, à réemployer lors de campagnes ultérieures

- **Objectif : définir les méthodes permettant la création de jeux de données**
- **Comment : trois méthodes principales**
 - **Saisie manuelle lors de l'exécution** : les données sont décrites dans les cas de test et créées par le testeur lors de l'exécution
 - **Par génération automatisée** : scripts SQL, outils de génération de données (des développements spécifiques peuvent être nécessaires)
 - **Par extraction ou copie de données de production** : à la demande ou régulièrement ; données actuelles ou passées.
- **Bonnes pratiques**
 - Identifier une méthode aussitôt que possible, surtout si des développements sont nécessaires
 - Examiner les besoins des autres AMOA afin de coordonner la génération automatisée ou la copie de données de production durant les tests métier
 - Etre attentif à la compatibilité des données entre différents environnements de test

- **Objectifs : élaborer des scénarii métier reposant sur la cible métier**
 - Processus / activités / tâches et acteurs métier cibles
 - Ces scénarii seront déroulés avec des utilisateurs durant la campagne de tests métier
- **Comment**
 - Elaborer des cas d'utilisation métier reproduisant les tâches des utilisateurs finaux dans des contextes spécifiques
 - Les cas d'utilisation métier devront être représentatifs
 - Les cas d'utilisation métier valident généralement plusieurs exigences
- **Bonnes pratiques**
 - Se concentrer sur les tâches et les objectifs des utilisateurs, pas sur les fonctions applicatives
 - Définir avec les utilisateurs clés les cas d'utilisation métier pertinents

2. Comment?

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.1.1. Définir quoi tester

2.1.2. Définir le périmètre de test

2.1.3. Concevoir les cas de test

2.1.4. Contrôler les ressources de test

2.1.5. Formaliser un plan de test

2.1.3. Contrôler les ressources de test

Resource	A vérifier
Ressources humaines • <i>Autres AMOA</i> • <i>Acteurs métier</i> • <i>Acteurs SI</i>	• Vérifier la disponibilité des acteurs
Environnements et outils de tests	• Vérifier la disponibilité des outils et des environnements de test
Matériel • <i>Postes de travail spécifiques</i> • <i>Appareils mobiles</i>	• Vérifier la disponibilité du matériel
Référentiel de tests	• <i>Cas de test existants, à réutiliser, à mettre à jour ou à supprimer</i>

En cas de ressources indisponibles :

- Identifier les tests non exécutables et les contournements possibles
- En l'absence de contournements possibles, obtenir les consignes ou la validation de la direction

- **Objectif : valider la couverture existante des besoins en environnements et outils de test**
- **Comment**
 - Vérifier :
 - La pertinence et la disponibilité des jeux de données
 - Les paramètres métier et les droits utilisateurs
 - Les interfaces applicatives
 - Les interfaces avec des applications sur d'autres environnements de test
 - Les bouchons simulant des interfaces
 - Caractéristiques et contraintes techniques
 - Disponibilité
 - Performance
- **Bonnes pratiques**
 - Valider dès que possible les besoins en environnements de tests (besoins potentiels en développement)

Dès que possible, prendre en compte les contraintes et limites de l'environnement de test

- **Valider la disponibilité de l'environnement de test**
 - Une autre campagne de test est-elle prévue en même temps ?
 - Si oui, les besoins en jeux de données sont-ils compatibles ?
 - Si oui, peut-on partager les environnements / les données sans chevauchement (sans risques d'effets secondaires sur les tests) ?
- **Connaître les limites de l'environnement de tests métier**
 - Peut-on obtenir des données de production (quantité importante de données pertinentes) ?
 - Si des tests multi-applications doivent être exécutés
 - Existe-t-il des bouchons pour simuler des messages ou peut-on utiliser des interfaces de test ?
 - Les jeux de données des différentes applications sont-ils compatibles ?
 - Les AMOA des autres applications ont-ils été informés des tests prévus ?

- **Objectif : partager un répertoire contenant tous les cas de test**
- **Le répertoire change suite aux nouveaux projets :**
 - De nouveaux tests sont ajoutés
 - Les tests obsolètes sont supprimés
 - Des tests existants sont modifiés
- **Les tests de non-régressions sont maintenus et réemployés pour différents projets**
- **Un référentiel de test bien entretenu est un actif permettant**
 - De gagner du temps sur la définition des tests (par exemple en cas de mise à jour technique d'un logiciel)
 - Aux nouveaux venus de s'adapter

2. Comment?

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.1.1. Définir quoi tester

2.1.2. Définir le périmètre de test

2.1.3. Concevoir les cas de test

2.1.4. Contrôler les ressources de test

2.1.4. Formaliser un plan de test

2.1.4. Formaliser un plan de test

- **Objectifs**
 - Vérifier la pertinence de la couverture des exigences et des processus métier cible par les tests métier
 - Valider que les limites (en raison de l'indisponibilité des ressources de test) sont connues et approuvées
 - Valider une charge de travail réaliste
- **Contenu principal**
 - Processus métier et cas d'utilisation métier cibles
 - Listes des exigences
 - Liste des cas de test
 - Matrice de couverture des exigences
 - Définition des jeux de données
 - Ressources de test

2. How

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

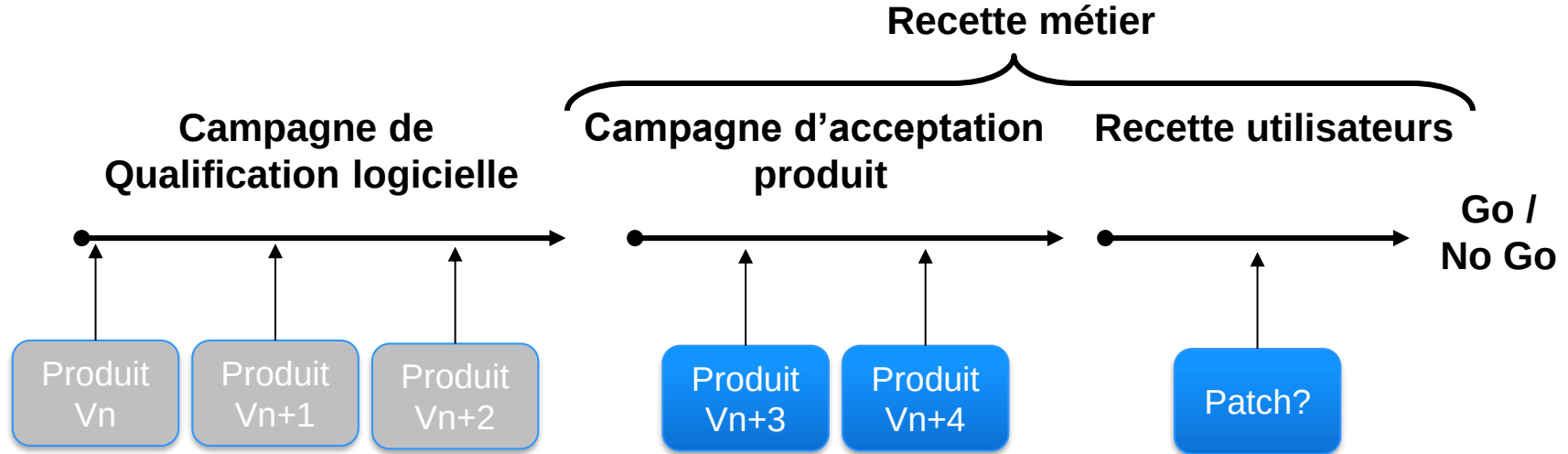
2.2.1. Préparer une campagne de test

2.2.2. Exécuter une campagne de test

2.2.1. Préparer une campagne de tests

Campagnes de tests et cycles

2.2. Exécuter une campagne de test



- Une campagne de tests de qualification ou de recette métier **comprend des cycles de livraison intermédiaires**
 - Livraisons itératives de nouvelles fonctions
 - Livraisons de réparations d'anomalies
- **Si les tests d'acceptation Produit sont concluants**, la dernière livraison de la campagne de recette métier sera celle **déployée en production**
 - Chaque fonctionnalité est livrée et testée
 - Les anomalies résiduelles sont connues et acceptées

2.2.1. Préparer une campagne de tests

Tests à exécuter

2.2. Exécuter une campagne de test

Tests de recevabilité

- Se concentrer sur les fonctions clés (fait en quelques heures, une journée au maximum)
- L'objectif est d'assurer un bon démarrage de campagne

Tests des nouvelles exigences

- Exécuter les tests par ordre de priorité des exigences
- Rejouer les tests les plus critiques à chaque cycles (pour éviter les régressions)

Tests de non régression

- Les tests de non régression se concentrent sur les exigences critiques en cours
- Exécuter les plus critiques dès la première itération
- La charge de travail peut être énorme

Tests des corrections d'anomalie

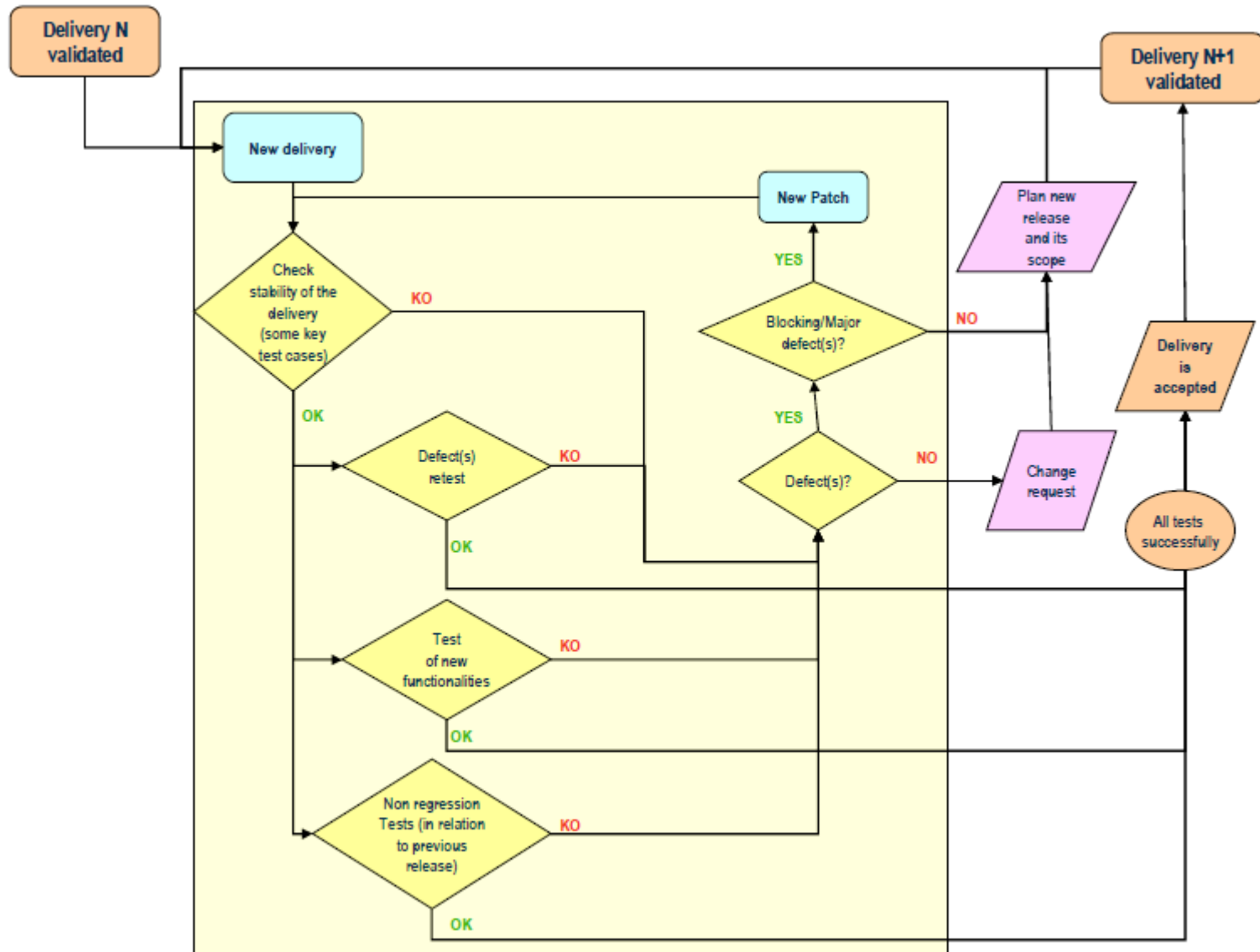
- Demander un patch rapidement (avant la fin du cycle en cours) uniquement pour les anomalies les plus bloquantes
- Si les développeurs ne peuvent les corriger toutes, les prioriser

Une campagne de tests de qualification et de tests métier comporte généralement ces quatre types de tests

2.2.1. Préparer une campagne de tests

Tests à exécuter : exemple

2.2. Exécuter une campagne de test



2.2.1. Préparer une campagne de tests

Tests à exécuter : tests de non régression

2.2. Exécuter une campagne de test

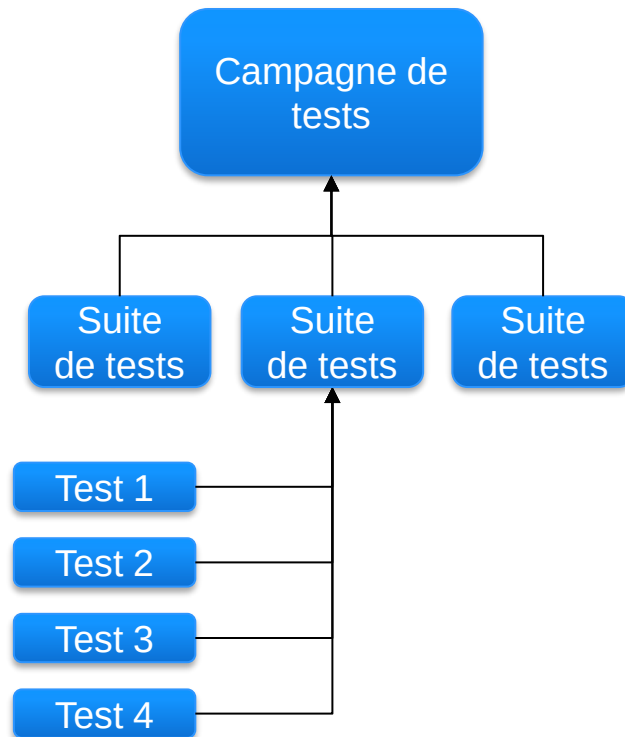
- **Objectif des tests de non régression**
 - Vérifier qu'aucune régression ne sera livrée en production
 - Vérifier que les exigences critiques existantes sont toujours couvertes par la mise à jour du logiciel
- **Les tests de non régression critiques doivent être exécutés à chaque livraison**
 - Indépendamment des évolutions
 - Centrés sur les fonctions critiques de l'application
- **Certains tests de non régression seront sélectionnés en raison des évolutions**
 - Les fonctions importantes pouvant être impactées par cette évolution
- **Critères de sélection des tests de non régression**
 - Criticité des exigences existantes
 - Stabilité de l'application
 - Complexité de la solution SI

- **Prévoir des sessions de test avec des acteurs métier**
 - Utilisateurs clé et / ou utilisateurs finaux
 - Vérifier que les utilisateurs impliqués dans les tests métier ont été suffisamment informés des changements prévus (conduite du changement)
- **Valider la disponibilité des jeux de données**
 - Objectif : jeux de données réalistes
 - Si ce n'est pas possible, expliquer clairement les limites
- **Valider les critères d'acceptation métier**
 - Ils reposeront sur les exigences et sur les cas d'utilisation métier
- **Déroulement des sessions**
 - Partager (ou non) les cas d'utilisation métier entre les utilisateurs
 - Décider de l'ordre de passage des cas d'utilisation

2.2.1. Préparer une campagne de tests

Définir des suites de tests

2.2. Exécuter une campagne de test



- Une suite de tests est un sous-ensemble d'une campagne de tests
- Il s'agit de préparer l'exécution en regroupant les tests à exécuter ensemble
- Selon les besoins / contextes, les tests seront regroupés selon différents critères :
 - Par exigences
 - Par types de tests (recevabilité, nouvelles exigences, non régression)
 - Par acteurs concernés par la campagne
 - Par priorité d'exécution
 -

2. How

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.2.1. Préparer une campagne de test

2.2.2. Exécuter une campagne de test

2.2.2. Exécuter une campagne de tests

Tests d'acceptation produit : principes

2.2. Exécuter une campagne de test

- Une campagne sera structurée selon les 3 principes suivants :

Principes	Objectifs
• Commencer par tester ce qui est critique, ou les prérequis pour tester ce qui est critique	• Trouver les anomalies majeures dès que possible
• Limiter autant que possible le nombre de livraisons	• Réduire la charge (procédure de livraison, tests de recevabilité et de non régression)
• Concentrer l'effort de test sur le début de la campagne	• Trouver autant d'anomalies que possible dès que possible

Tout au long de l'exécution la campagne de tests s'adaptera aux aléas du projet et à la quantité d'anomalies découvertes

2.2.2. Exécuter une campagne de tests

Tests d'acceptation produit : exemple

2.2. Exécuter une campagne de test

Tests	Cycle 1	Cycle 2	Cycle 3
Tests de recevabilité	✓	✓	✓
Tests des nouvelles exigences - Critique - Important - Moins important	✓ ✓ ✓	✓	✓ ✓
Tests de non régression - Critique - Important	✓	✓ ✓	✓
Tests de correction des anomalies		✓	✓

- **Communiquer sur les anomalies résiduelles**
 - Vérifier que les activités métier peuvent être réalisées
 - Définir ou proposer des contournements
- **Communiquer sur les limites liées à l'environnement de test**
- **Exécuter les cas d'utilisation métier**
 - Décrire les nouvelles anomalies et évaluer leurs impacts métier et leurs gravités
 - Indiquer les carences et évaluer leurs impacts métier
 - Définir des contournements
- **Pièges à éviter**
 - Les utilisateurs ont parfois tendance à préférer l'existant à la cible
 - Les utilisateurs ont parfois tendance à exagérer la gravité d'une anomalie

2. How

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.3.1. Décrire une anomalie

2.3.2. Résoudre une anomalie

2.3.1. Décrire une anomalie

Anomalies et carences

Anomalie

- Non-conformité **avec le comportement attendu** (exigences, spécifications)
- Doit être corrigé (selon la gravité)
- Peut mener à un contournement

Carence

- Non-conformité **imprévue**
- Conduit en un nouveau cycle de développement logiciel
- Peut mener à un contournement

La gestion des carences est un point clé de la recette métier

2.3.1. Décrire une anomalie

Attributs : description

- **Objectif : décrire précisément une anomalie permet de la corriger avec pertinence et efficacité**
 - Plus la description sera précise et exacte, plus aisée sera la correction par les développeurs
- **Comment : la description de l'anomalie**
 - Situation au démarrage : état du logiciel, jeux de données
 - Etapes de test réalisées avant que l'anomalie ne se produise
 - Ce qui était attendu quand l'anomalie s'est produite
 - Ce qui s'est passé
- **Bonnes pratiques**
 - Reproduire l'anomalie
 - Vérifier que ce n'est pas une erreur humaine
 - Vérifier que ce n'est pas une carence
 - Joindre des copies d'écran avec des explications

2.3.1. Décrire une anomalie

Attributs : gravité

- **Objectif : améliorer l'évaluation de la gravité de l'anomalie**
 - Gravité de l'anomalie : impact sur les activités métier si l'anomalie se produit en contexte opérationnel
 - Critères clé pour définir les priorités et décider du Go Live
- **Comment : attribuer les valeurs de gravité habituelles**
 - *Critique / majeur* : le processus / l'activité métier ne peut aboutir à cause de l'anomalie, et il n'y a pas de contournement
 - *Moyen* : un contournement est possible, mais il pénalise les utilisateurs
 - *Mineure / bénigne* : les utilisateurs peuvent travailler, mais une correction serait appréciée
- **Bonnes pratiques**
 - Valider l'accord des **acteurs métier impactés** par l'anomalie concernant sa gravité
 - Déterminer les **impacts métier** de l'anomalie : les conséquences d'un point de vue métier de l'occurrence de l'anomalie
 - Evaluer sa **fréquence**
 - Valider les **contournements** potentiels

2.3.1. Décrire une anomalie

Attributs : catégorie et type

- **Objectif : pouvoir présenter les anomalies résiduelles d'un point de vue métier à la fin de la phase de tests métier**
 - Différence entre ancienne et nouvelle version en production
 - Différences entre anomalies résolus, nouvelles anomalies et régression d'un point de vue métier
- **Comment**
 - Evaluer la **catégorie de l'anomalie** : distinguer les anomalies d'acceptation et de production
 - Les anomalies de production doivent être résolues, mais ne sont généralement pas significatives pour décider du Go Live
 - Les anomalies d'acceptation résiduelles doivent être évaluées avant le Go Live. Celles qui ne sont pas résolues deviennent des anomalies de production
 - Evaluer le **type de l'anomalie** : distinguer nouvelles anomalies et régressions
 - Nouvelle anomalie : une anomalie concernant les nouvelles fonctionnalités
 - Régression: une anomalie concernant les anciennes fonctionnalités

2.3.1. Décrire une anomalie

Attributs

Attributs	Objectif
ID	• Identifier avec certitude une anomalie particulière
Titre	• Doit suffire pour comprendre ce qui a échoué
Produit	• Utile pour les solutions SI multi-application
Description	• Décrire précisément une anomalie permet une correction pertinente et efficace
Statut	• Suivre le cycle de vie de l'anomalie (nouvelle, rejetée, ouverte, résolue, fermée)
Acteurs impactés	• Acteurs métier touchés par l'anomalie
Impact métier	• Impact de l'anomalie sur les activités métier
Gravité métier	• Evaluation reposant sur les acteurs touchés et sur l'impact métier
Priorité de résolution	• Priorité donnée à la MOE pour la livraison d'une réparation
Catégorie	• Anomalie d'acceptation ou de production
Type	• Nouvelle anomalie ou régression
Assigné à	• Acteur actuellement chargé de l'anomalie
Détecté dans la version	• Savoir dans quelle version de l'application l'anomalie se produit
Version prévue pour la résolution	• Version livrée par la MOE dans laquelle l'anomalie est résolue
Résolue dans la version	• Connaître la version de l'application dans laquelle l'anomalie est fermée

2. How

2.1. Concevoir un plan de test

2.2. Exécuter une campagne de test

2.3. Gérer des anomalies

2.4. Faire un bilan de recette

2.3.1. Décrire une anomalie

2.3.2. Résoudre une anomalie

2.3.2. Résoudre une anomalie

Suivre la correction des anomalies par la MOE

- **Objectif : résoudre les anomalies selon leur priorité**
 - Une anomalie empêchant la poursuite des tests a la plus haute priorité et implique la livraison d'un patch au plus tôt
- **Comment**
 - Valider les anomalies avec la MOE (reproductibilité possible, résultat attendu indiqué clairement)
 - Répondre aux questions / remarques de la MOE durant l'analyse
 - Obtenir de la MOE une date prévisionnelle pour la livraison du patch
- **Bonnes pratiques**
 - Planifier la livraison des versions de résolution de l'application (patches potentiels attendus)
 - Planifier une réunion hebdomadaire avec la MOE pour examiner les anomalies
 - Eviter les mails pour analyser les anomalies avec la MOE

2.3.2. Résoudre une anomalie

Retester une anomalie corrigée

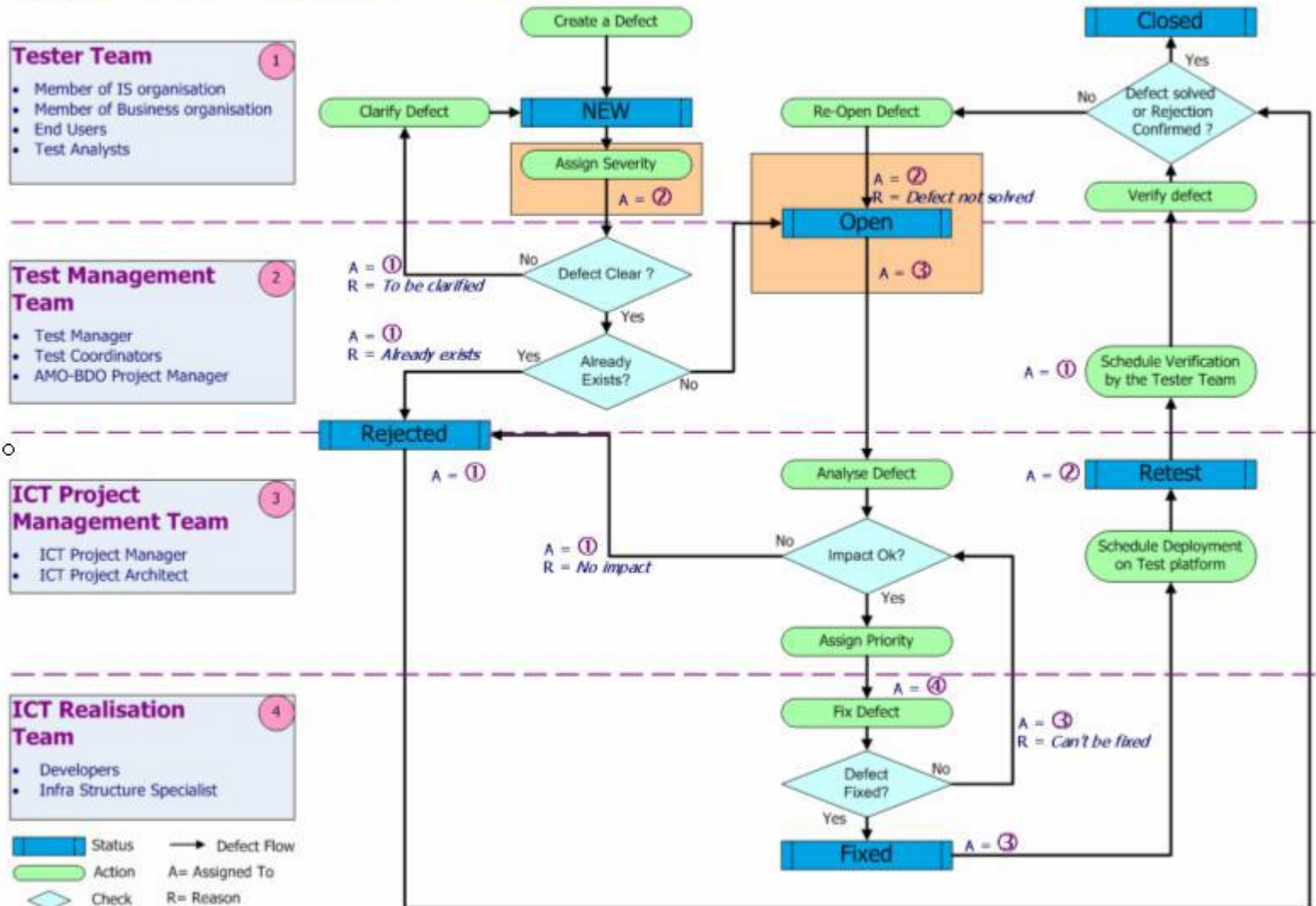
- **Objectif** : valider qu'une anomalie est vraiment résolue et qu'elle ne provoque pas de nouvelles anomalies
- **Comment**
 - Réexécuter le test dans les mêmes conditions pour vérifier que l'anomalie est vraiment résolue ; si c'est le cas, fermer l'anomalie ; sinon, la rouvrir
 - Achever le cas de test, si l'anomalie se produisait lors d'un pas de test intermédiaire ; ouvrir une nouvelle anomalie, si le cas de test échoue plus tard.
- **Bonnes pratiques**
 - Indiquer la version de résolution d'une anomalie : permet de faire des statistiques sur les temps de résolution.

2.3.2. Résoudre une anomalie

Un exemple de Workflow

2.3. Gérer des anomalies

Symphony – Defect Management Lifecycle



2. Comment

- 2.1. Concevoir un plan de test
- 2.2. Exécuter une campagne de test
- 2.3. Gérer des anomalies
- 2.4. Faire un bilan de recette**

- **Objectif** : permettre au métier de statuer sur le Go/No go pour un usage en production d'une solution SI
- **Comment** : périmètre
 - Liste des exigences couvertes par la livraison SI
 - Liste des applications concernées par la livraison
- **Bonnes pratiques**
 - Livrer une matrice de couverture des exigences avec les tests réellement exécutés (et pas les tests prévus)

- **Objectif : permettre au métier de statuer sur le Go/No go pour un usage en production d'une solution SI**
- **Comment : limites**
 - Signaler ce qui n'a pas été testé et pourquoi (contraintes liées à l'environnement, cas particuliers...)
 - Proposer un plan de mitigation (tester en production si nécessaire, pilote après le go live...)
- **Bonnes pratiques**
 - Analyse de risques

- **Objectif** : permettre au métier de statuer sur le Go/No go pour la mise en production d'une solution SI
- **Comment** : recenser les anomalies

Contenu	Objectif
• Anomalies résiduelles sur les nouvelles exigences avec leur gravité	• Vérifier que la nouvelle solution SI couvre suffisamment les besoins métier
• Anomalies de production résolues	• Liste des améliorations apportées par la solution
• Régressions avec leur gravité	• Evaluer la criticité des régressions

- **Bonnes pratiques**
 - Utiliser les **catégories** et les **types** des anomalies
 - Ne pas mentionner les anomalies découvertes et fermées durant les tests métier
 - Pour le métier, les régressions sur les nouvelles fonctionnalités ne sont pas des régressions

- **Bonnes pratiques**

- Autant que possible, cadrer les **impacts métier** de l'anomalie (fréquence, conséquences pour le métier, contournements)
- Autant que possible, s'il y a plusieurs acteurs métier, classer les anomalies par **acteur touché**